



PROJECT REPORT

SUBJECT:

COMPUTER ORGANIZATION &
ASSEMBLY LANGUAGE

PROGRAM:

BS DATA SCIENCE

SUBMITTED TO:

SIR ASAD

SIR ADEEL

PROJECT NAME:

ASCII-Based Encryption Utility

SUBMITTED BY:

FIZZA FAROOQ

MARYAM SHARIF

ROLL NUMBER:

SU92-BSDSM-F23-017

SU92-BSDSM-F23-038

BSDS (6A)



TABLE OF CONTENTS

1. Introduction	
2. Objectives	
3. Tools & Technologies Used	
4. System Overview	
5. Algorithm Description	
5.1 Encryption Algorithm	
5.2 Decryption Algorithm	
6. Core Concept Used.....	
7. Text Processing	
8. Technical Implementation Details	
8.1 Input Handling	
8.2 Key Handling	
8.3 Output Handling	
9. Features.....	
10. Limitations	
11. Future Enhancements	
12. Conclusion	
13. OUTPUTS.....	



ASCII-Based Encryption Utility (Assembly Language)

1. Introduction

This project is an assembly language-based encryption and decryption utility developed using **8086 microprocessor instructions**. It implements the classical **Caesar Cipher technique**, where each character in the input text is shifted by a numeric key value. The tool provides a simple interactive menu that allows the user to select encryption or decryption, enter text, and apply a key value ranging from **0 to 25**.

2. Objective

The main objectives of this project are:

- To understand low-level programming using **8086 Assembly Language**
- To implement a basic **cryptographic algorithm (Caesar Cipher)**
- To handle **string input/output using DOS interrupts**
- To allow dynamic key input (**0–25 range**)
- To design a simple menu-driven interface in assembly

3. Tools & Technologies Used

- Assembly Language (8086)
- DOSBox (for execution)
- DOS Interrupts (**int 21h**)
- ASCII-based character manipulation
- NASM Assembler

4. System Overview

The system works in the following steps:

1. Display welcome message and menu
2. User selects:
 - Encrypt text
 - Decrypt text
3. User enters input string
4. User enters key (**0–25**)
5. Program processes each character:
 - Shifts characters for encryption/decryption



6. Displays final result
7. Option to return to menu or exit

5. Algorithm Description

Encryption Algorithm:

- Read input text
- For each character:
 - Add key value
 - Apply ASCII boundary checks
 - Wrap around alphabet if needed

Decryption Algorithm:

- Read encrypted text
- For each character:
 - Subtract key value
 - Apply reverse wrapping logic

6. Core Concepts Used:

- 8086 Assembly Language Programming
- DOS Interrupts (INT 21h) for input and output operations
- Menu-driven program structure (Encrypt / Decrypt selection)
- Conditional jumps for decision making (encrypt vs decrypt, character checks)
- Looping for repeated input processing and program repetition
- String handling for text input, processing, and output display
- ASCII code manipulation for alphabetic characters
- Arithmetic operations for Caesar cipher shifting (add, subtract, modulo 26)
- Register usage (AX, BX, CX, DX, SI) for data processing and indexing
- Memory variables for data storage (text buffer, key, user choice)
- Case-sensitive handling of uppercase and lowercase characters
- Input validation logic (non-alphabet characters are skipped)

7.TEXT PROCESSING

Character Handling:

- Read each character from buffer until \$ (string terminator).
- Check character type:
 - A-Z → Uppercase processing



- a–z → Lowercase processing
- Others → skipped (no change)

Uppercase Processing (A–Z):

- Convert to 0–25 range:

$$AL = AL - 'A'$$

- Apply operation:
 - Encrypt: $AL = AL + \text{key}$
 - Decrypt: $AL = AL - \text{key}$
- Handle negative values:

$$AL = AL + 26$$

- Apply modulo:

$$AL = AL \% 26$$

- Convert back to ASCII:

$$AL = AL + 'A'$$

Lowercase Processing (a–z):

- Convert to 0–25 range:

$$AL = AL - 'a'$$

- Apply operation:
 - Encrypt: $AL = AL + \text{key}$
 - Decrypt: $AL = AL - \text{key}$
- Handle negative values:

$$AL = AL + 26$$

- Apply modulo:

$$AL = AL \% 26$$

- Convert back to ASCII:

$$AL = AL + 'a'$$



8. Technical Implementation Details

Input Handling

- Uses int 21h function 01h for character input
- String stored in buffer until Enter key is pressed

Key Handling

- Multi-digit key conversion using multiplication logic:
 - $\text{key} = (\text{key} * 10) + \text{digit}$

Output Handling

- Uses int 21h function 09h for string display

9.Features:

No.	Feature	Description
1	Program Header	Displays “CAESAR CIPHER TOOL” title at startup
2	Menu System	Provides options for Encrypt and Decrypt
3	Text Input	Accepts user input string character by character
4	Key Input	Takes numeric key (0–25) from user
5	Key Conversion	Converts multi-digit key into usable number
6	Encryption	Shifts letters forward using Caesar Cipher logic
7	Decryption	Shifts letters backward using same key
8	Uppercase Handling	Encrypts/decrypts A–Z separately
9	Lowercase Handling	Encrypts/decrypts a–z separately
10	Non-alphabet Handling	Keeps symbols, spaces unchanged
11	Modulo Logic	Ensures wrap-around within 26 alphabets
12	Result Display	Shows final encrypted/decrypted text
13	Loop System	Allows returning to main menu
14	Exit System	Ends program using DOS interrupt

10.Limitations

- Only supports ASCII characters
- No special character encryption logic
- Fixed buffer size (60 characters)



- No file handling support

11. Future Enhancements

- File-based encryption/decryption
- GUI-based interface
- Stronger encryption algorithms (AES/RSA simulation)
- Support for Unicode characters
- Increased buffer size and dynamic memory handling

12. Conclusion

The Caesar Cipher Tool successfully demonstrates the fundamentals of assembly language programming, memory manipulation, and basic cryptography. It helps in understanding how low-level instructions control data processing at the hardware level. This project also strengthens concepts of loops, condition checking, interrupts, and register operations in 8086 architecture.

OUTPUT

Encryption:

```
C:\>nasm coal.asm -o coal.com  
C:\>coal.com
```

```
=== CAESAR CIPHER TOOL ===  
1.Encrypt 2.Decrypt  
Choice: 1  
Enter text: fizza  
Enter key (0-25): 17  
Result: wzqqr
```

Decryption:

```
1.Encrypt 2.Decrypt  
Choice: 2  
Enter text: wzqqr  
Enter key (0-25): 17  
Result: fizza  
Back to menu? (y/n): n
```